Node.js process 模块全解

掌握 Node.js 应用的 “仪表盘” 与 “控制台”

1. 认识 process：Node.js 的 “当前进程控制面板”

想象一下，如果不只是在写代码，而是在驾驶一架飞机，那么 `process` 对象就是你面前的**仪表盘和控制杆**。在 Node.js 中，每个运行的应用程序都是操作系统中的一个 “进程”（Process）。

`process` 是一个全局对象（Global Object），这意味着你不需要使用 `require('process')` 就能在任何地方直接使用它。它提供了当前 Node.js 进程的有关信息，并允许你控制该进程。

通过它，你可以实现以下核心功能：

- **环境感知**：读取环境变量（配置数据库、API 密钥）。
- **输入输出**：处理命令行参数，与终端进行读写交互。
- **生命周期**：监听退出事件，或者主动结束程序。
- **状态监控**：查看内存占用、运行时间。

更多详细 API 定义可参考 Node.js 官方文档。

1.1 快速上手：5 分钟小示例

让我们通过一个最小的可运行示例，一次性体验 `process` 的几个核心属性。创建一个名为 `demo.js` 的文件：

demo.js

```

// 1. 读取环境变量 (尝试读取 USER 或 USERNAME)
const user = process.env.USER || process.env.USERNAME || 'Guest';
console.log(`Hello, ${user}!`);

// 2. 获取命令行参数
const args = process.argv.slice(2); // 去掉前两个默认参数
console.log(`收到参数: ${args.join(', ')}`);

```

```
// 3. 打印进程基本信息
console.log(`------------------`);
console.log(`PID (进程 ID): ${process.pid}`);
console.log(`Platform (平台): ${process.platform}`);
console.log(`CWD (工作目录): ${process.cwd()}`);
console.log(`------------------`);

// 4. 优雅退出
if (args.includes('--fail')) {
    console.error('遇到错误，非正常退出...');
    process.exit(1);
} else {
    console.log('任务完成，正常退出。');
    process.exit(0);
}

```

运行方式：

终端执行

```

# 正常运行
node demo.js one two

# 模拟错误退出
node demo.js --fail

```

1.2 进程信息：我是谁、在哪儿、运行多久

了解当前进程的元数据对于日志记录和跨平台兼容性至关重要。以下是常用的身份信息属性：

属性/方法	用途说明	示例输出
`process.pid`	当前进程的数字 ID，用于系统管理	`12345`
`process.ppid`	父进程 ID (Parent PID)	`12344`
`process.platform`	操作系统平台	`'darwin'`, `'win32'`, `'linux'`
`process.arch`	CPU 架构	`'x64'`, `'arm64'`
`process.version`	Node.js 版本号	`'v18.16.0'`

`process.uptime()`	进程已运行时间（秒）	`3.1415`

注意： `process.platform` 在 macOS 上返回 `'darwin'` ，在 Windows 上返回 `'win32'` （即使是 64 位系统）。编写跨平台脚本时请务必判断此属性。

1.3 环境变量：配置你的应用

`process.env` 是最常用的属性之一，它返回一个包含用户环境信息的对象。

> **最佳实践：默认值与类型转换**
>
> 环境变量读取到的值**永远是字符串**。如果是数字或布尔值，必须手动转换。同时，建议始终提供默认值以防配置缺失。

env_demo.js

```
1
2   // 错误做法：直接使用，未处理 undefined 或类型
3   // const port = process.env.PORT;
4
5   // 正确做法：
6   const PORT = parseInt(process.env.PORT || '3000', 10);
7   const IS_DEBUG = process.env.NODE_ENV === 'development';
8   const API_KEY = process.env.API_KEY;
9
10  if (! API_KEY) {
11    console.warn('警告：未配置 API_KEY，部分功能可能不可用');
12  }
13
14  console.log(`服务启动于端口：${PORT}，调试模式：${IS_DEBUG}`);
15
```

如何设置环境变量（跨平台）：
在 Windows (cmd) 和 Unix (Bash) 下设置变量的语法不同。为了统一开发体验，推荐使用 cross-env 库。

package.json script

```
1
2   "scripts": {
3     "start": "cross-env NODE_ENV=production PORT=8080 node app.js"
4   }
5
```

1.4 命令行参数：让程序可配置

`process.argv` 是一个数组，包含启动 Node.js 进程时的命令行参数。

原生结构解析：

- `argv[0]`：Node.js 可执行文件的绝对路径。
- `argv[1]`：正在执行的脚本文件的绝对路
- `argv[2...]`：真正的用户参数。

常见坑：

如果参数中包含空格（如文件路径 `My Files`），在命令行输入时必须加引号，否则会被识别为两个参数。

对于复杂的 CLI 工具（如处理 `--help`，`-v`），手动解析 `argv` 会非常繁琐。推荐使用 **Commander.js** 或 **Yargs** 等成熟库。

1.5 标准输入输出：与终端对话

Node.js 提供了三个标准流：`process.stdin`（输入）、`process.stdout`（输出）、`process.stderr`（错误）。

`console.log` 内部其实就是调用的 `process.stdout.write`（加上了换行符）。使用标准流可以实现更有趣的交互，比如管道操作。

pipe_demo.js

```

// 演示：将输入的数据转换为大写后输出
process.stdin.setEncoding('utf8');

process.stdin.on('data', (chunk) => {
    const input = chunk.trim();
    if (input === 'exit') process.exit(0);

    // 写入标准输出
    process.stdout.write(`Echo: ${input.toUpperCase()}\n`);
});

process.stderr.write('应用已启动，请输入内容（输入 exit 退出）:\n');

```

管道玩法： 你可以将一个程序的输出直接传给 Node.js。

Code block

```

echo "hello node" | node pipe_demo.js
# 输出: Echo: HELLO NODE

```

1.6 退出与返回码：优雅结束程序

当程序完成任务或遇到不可恢复的错误时，需要终止进程。

1. **process.exit(code)**: 强制立即退出。
   `code=0` 表示成功；`code=1`（或非 0）表示出错。

2. **exit 事件**: 临终遗言。
   当进程即将退出时触发。**注意：** 此回调中只能执行同步代码，异步操作（如写数据库）将被直接截断放弃。

exit_handling.js

```
process.on('exit', (code) => {
    // 只能做同步清理，例如删除临时文件
    console.log(`进程即将结束，退出码: ${code}`);
});

try {
    // 模拟业务逻辑
    throw new Error('严重错误');
} catch (err) {
    console.error(err.message);
    process.exit(1); // 告诉系统出错了
}

```

1.7 进程信号：优雅处理中断与终止

当你在终端按下 `Ctrl+C` 时，系统会向进程发送 `SIGINT` 信号。默认行为是直接终止，但我们可以监听它来实现“优雅退出”（Graceful Shutdown），比如关闭数据库连接。

signal.js

```
console.log('程序运行中... 按 Ctrl+C 尝试退出');

// 保持程序不退出
setInterval(() => {}, 1000);

process.on('SIGINT', () => {
    console.log('\n 收到 SIGINT 信号。正在清理资源...');

    // 模拟异步清理（例如关闭 DB 连接）
```

```
    setTimeout(() => {
        console.log('清理完毕。再见! ');
        process.exit(0);
    }, 500);
});

```

注意： `SIGKILL` 和 `SIGSTOP` 信号是无法被监听或拦截的，它们会强制杀死进程。

1.8 性能与内存：量化你的程序状态

想知道代码跑得有多快？内存有没有泄露？ `process` 提供了高精度的工具。

1.8.1 1. 高精度计时 (hrtime)

Code block

```

const start = process.hrtime.bigint();

// 执行耗时操作
for (let i = 0; i < 1e6; i++) {}

const end = process.hrtime.bigint();
// 结果是纳秒 (nanoseconds)，除以 1e6 转换为毫秒
console.log(`耗时: ${Number(end - start) / 1e6}ms`);

```

1.8.2 2. 内存监控 (memoryUsage)

Code block

```

const used = process.memoryUsage();
console.log({
    rss: `${Math.round(used.rss / 1024 / 1024)} MB`, // 驻留集大小 (总物理内存)
    heapUsed: `${Math.round(used.heapUsed / 1024 / 1024)} MB`, // JS 对象占用的堆内存
    external: `${Math.round(used.external / 1024 / 1024)} MB` // C++ 对象绑定内存
});

```

1.9 工作目录：别把 “家” 弄丢了

新手常混淆 `process.cwd()` 和 `__dirname`。

- **process.cwd(): 当前工作目录**（Current Working Directory）。即你在终端敲下 `node` 命令时所在的文件夹。
- **__dirname: 当前代码文件所在目录**。这是固定不变的。

你可以使用 `process.chdir()` 改变工作目录，但这通常不仅危险（影响后续文件读取相对路径），而且容易在多线程/异步环境下产生竞态条件，请谨慎使用。

1.10 最佳实践与避坑清单

- **不要在 exit 事件中写异步代码**：它不会被执行。清理逻辑应放在 `SIGINT` 或自定义 shutdown 函数中。
- **正确使用 stdout/stderr**：正常的业务输出用 `console.log` (stdout)，错误日志一定要用 `console.error` (stderr)，方便日志系统分流。
- **处理未捕获异常**：虽然可以监听 `uncaughtException` 防止崩溃，但这会导致程序状态不可控。最佳做法是：记录错误 → 优雅关闭连接 → `process.exit(1)` 重启。
- **不要硬编码敏感信息**：永远不要在代码里写 `process.env.PASSWORD = '123'`，应使用 `.env` 文件配合 `dotenv` 库。

1.11 速查表：日常开发翻一翻

API	分类	一句话说明
`process.env`	环境	读取系统环境变量
`process.argv`	输入	获取命令行参数数组
`process.cwd()`	路径	获取当前执行命令的目录
`process.exit(code)`	控制	结束进程 (0=成功， 1=失败)
`process.on('SIGINT', fn)`	事件	拦截 Ctrl+C 中断信号
`process.nextTick(fn)`	异步	在当前操作完成后立即执行回调
`process.memoryUsage()`	监控	查看内存使用详情